Ascent

FORMATION

FORMATION PROFESSIONNELLE · COURS APPRENANT

Cours apprenant

ReST API — bonnes pratiques et sécurité

MODULE 1

Introduction aux API REST

Comprendre les principes, les méthodes HTTP et les bonnes pratiques de conception

CRÉATEUR DU COURS

Boris Rose

Formateur & concepteur du contenu

Table des matières

Avant-propos **3**

1 Qu'est-ce qu'une API ? **3**
1.1 Définition . . . 3
1.2 Trois exemples du quotidien . . . 3
1.3 Le contexte historique . . . 3

2 REST : un style architectural **4**
2.1 Définition . . . 4
2.2 Les 6 contraintes de REST . . . 4
2.3 Trois piliers conceptuels . . . 4
2.4 Définitions associées . . . 4

3 Anatomie d'une requête et d'une réponse HTTP **5**
3.1 La requête HTTP . . . 5
3.2 La réponse HTTP . . . 5
3.3 Les en-têtes essentiels . . . 6

4 Les méthodes HTTP utilisées en REST **6**
4.1 GET . . . 6
4.2 POST . . . 6
4.3 PUT . . . 7
4.4 PATCH . . . 7
4.5 DELETE . . . 7
4.6 Tableau récapitulatif . . . 7
4.7 Notions importantes : « sûre » et « idempotente » . . . 7

5 Les codes de statut HTTP **7**
5.1 Les 5 familles . . . 8
5.2 Les codes 2xx les plus utilisés . . . 8
5.3 Les codes 4xx les plus utilisés . . . 8
5.4 Les codes 5xx . . . 8
5.5 Confusion classique : 401 vs 403 . . . 8

6 Bonnes pratiques de conception **8**
6.1 Nommer les URI . . . 8
6.2 Versionner l'API . . . 9
6.3 Pagination, filtrage, tri . . . 9
6.4 Gérer les erreurs : Problem Details . . . 9
6.5 Anti-patterns à éviter . . . 10

7 Les outils du quotidien **10**
7.1 Postman . . . 10
7.2 Insomnia . . . 10
7.3 cURL . . . 10
7.4 Swagger / OpenAPI . . . 11

8 Exemple complet : conception d'une API de blog **11**
8.1 Liste des endpoints . . . 11
8.2 Exemple de réponse paginée . . . 11

9 Synthèse **12**

10 Lexique du module **12**

11 Pour aller plus loin **13**

Avant-propos

Ce cours a été conçu pour que vous puissiez **revoir et approfondir** chez vous les notions vues en classe. Tous les termes techniques sont définis et tous les outils mentionnés sont décrits avec leur utilité. Lisez-le après la séance, en parallèle de vos travaux pratiques.

1 Qu'est-ce qu'une API ?

1.1 Définition

Définition

API : sigle anglais pour *Application Programming Interface*, soit « interface de programmation applicative ». Une API est un *contrat technique* entre deux logiciels : elle décrit comment l'un peut demander un service ou des données à l'autre, sans devoir connaître les détails internes de son implémentation.

L'API n'est ni le serveur ni la base de données : elle est l'*interface* qui se trouve entre le client et le serveur. Comme le menu d'un restaurant : on commande sans aller en cuisine, sans connaître la recette, en se contentant de respecter le format proposé.

1.2 Trois exemples du quotidien

Exemple

- Une application météo sur votre smartphone interroge l'API d'un fournisseur de données climatiques pour récupérer les prévisions.
- Le bouton « Se connecter avec Google » d'un site interroge l'API d'authentification de Google (*OAuth2*).
- Une plateforme de vente en ligne interroge l'API d'un transporteur (*La Poste, Chronopost*) pour suivre vos colis.

1.3 Le contexte historique

Les API web ont évolué en plusieurs vagues. Au début des années 2000, le standard dominant était **SOAP** (*Simple Object Access Protocol*), un protocole basé sur XML, très rigoureux mais lourd. En 2000, Roy Fielding (l'un des co-auteurs du protocole HTTP) propose dans sa thèse de doctorat un nouveau *style architectural* appelé **REST**, qui exploite mieux les capacités natives de HTTP. À partir de 2005, avec la montée du JSON et d'AJAX, REST s'impose comme le standard de fait pour les API publiques.

Plus récemment, d'autres approches sont apparues : **GraphQL** (Facebook, 2015) qui permet au client de décrire précisément les données qu'il veut, et **gRPC** (Google, 2016) qui privilégie la performance pour les communications inter-services. REST reste néanmoins majoritaire pour les API web publiques.

2 REST : un style architectural

2.1 Définition

Définition

REST (*REpresentational State Transfer*) est un *style architectural* défini par Roy Fielding en 2000. Ce n'est ni un protocole, ni un format, ni un framework : c'est un ensemble de *principes* qui, appliqués ensemble, permettent de bâtir des services web simples, scalables et évolutifs.

2.2 Les 6 contraintes de REST

Pour qu'une API mérite l'appellation REST, elle doit respecter six contraintes architecturales :

1. **Client–Serveur** : séparation stricte. Le client (l'interface utilisateur) et le serveur (la logique et les données) évoluent indépendamment.
2. **Sans état (stateless)** : chaque requête contient toute l'information nécessaire à son traitement. Le serveur ne « se souvient » pas du contexte d'une requête à l'autre.
3. **Cacheable** : les réponses indiquent explicitement si elles peuvent être mises en cache, et pour combien de temps.
4. **Système en couches** : le client ne sait pas s'il parle au serveur final, à un proxy ou à un cache. Cela permet d'introduire des intermédiaires (load balancer, CDN) sans casser le contrat.
5. **Interface uniforme** : toutes les ressources sont identifiées et manipulées de la même façon prévisible.
6. **Code à la demande (optionnel)** : le serveur peut envoyer du code exécutable au client, par exemple du JavaScript dans un navigateur.

2.3 Trois piliers conceptuels

- **Ressources** : tout ce que l'API expose est une « ressource » (un utilisateur, un article, une commande, un panier). Chaque ressource a un identifiant unique : son **URI**.
- **Représentations** : une ressource peut être renvoyée dans plusieurs formats (JSON, XML, HTML, CSV). Le client négocie le format via les en-têtes HTTP.
- **Verbes HTTP** : les opérations sur les ressources passent par les méthodes HTTP standard (GET, POST, PUT, PATCH, DELETE).

2.4 Définitions associées

Définition

URI (*Uniform Resource Identifier*) : chaîne de caractères qui identifie de manière unique une ressource. Une URI peut être absolue (`https://api.example.com/articles/42`) ou relative (`/articles/42`).

URL (*Uniform Resource Locator*) : sous-ensemble des URI qui inclut la *localisation* (le protocole et l'hôte). En pratique, on utilise les deux termes comme synonymes.

Définition

HTTP (*HyperText Transfer Protocol*) : protocole de communication standard du web, défini en 1989 par Tim Berners-Lee. Trois versions sont utilisées en 2026 :

- HTTP/1.1 : version historique, encore très répandue.
- HTTP/2 : multiplexage de plusieurs flux dans une seule connexion TCP.
- HTTP/3 : nouvelle génération basée sur QUIC (UDP), idéale pour les réseaux mobiles.

La sémantique d'une API REST est strictement la même quelle que soit la version de HTTP utilisée.

3 Anatomie d'une requête et d'une réponse HTTP

3.1 La requête HTTP

Une requête HTTP est constituée de quatre parties :

1. Une **ligne de démarrage** : la méthode HTTP, l'URI cible et la version du protocole.
2. Des **en-têtes (headers)** : des métadonnées sous forme de couples `Nom: valeur`.
3. Une **ligne vide** : qui sépare les en-têtes du corps.
4. Un **corps (body)** : optionnel, contient les données envoyées (JSON, formulaire, fichier).

Exemple

Exemple complet d'une requête de création d'article :

```
POST /api/v1/articles HTTP/1.1
Host: blog.example.com
Authorization: Bearer eyJhbGciOiJIUzI1NiIs...
Content-Type: application/json
Accept: application/json

{
  "title": "Introduction aux API REST",
  "author": "Boris Rose",
  "published": true
}
```

3.2 La réponse HTTP

Symétriquement, une réponse contient :

1. Une **ligne de statut** : version HTTP + code de statut + texte humain.
2. Des **en-têtes**.
3. Une **ligne vide**.
4. Un **corps** (optionnel).

Exemple

Réponse à la requête précédente :

```
HTTP/1.1 201 Created
Content-Type: application/json
Location: /api/v1/articles/42
```

```
{
  "id": 42,
  "title": "Introduction aux API REST",
  "createdAt": "2026-05-05T10:14:02Z"
}
```

3.3 Les en-têtes essentiels

- `Host` : nom de domaine du serveur visé.
- `Authorization` : authentification (`Bearer <token>`, `Basic ...`).
- `Content-Type` : format du corps (`application/json`, `text/html`...).
- `Accept` : format(s) attendu(s) en réponse.
- `Content-Length` : taille du corps en octets.
- `Cache-Control` : politique de cache.
- `Location` : URI de la nouvelle ressource après création.
- `ETag` : identifiant de version d'une ressource (utile pour le cache).

4 Les méthodes HTTP utilisées en REST

Chaque méthode HTTP a une intention sémantique précise.

4.1 GET

Définition

GET : lit une ressource. Cette méthode est *sûre* (sans effet de bord) et *idempotente* (N appels donnent le même résultat qu'un seul). Elle est aussi nativement *cacheable*.

- `GET /articles/42` : lire un article.
- `GET /articles` : lister tous les articles (avec pagination).
- `GET /articles?author=42&page=2` : filtrer et paginer via la *query string*.

Règle absolue. Un GET ne doit *jamais* modifier de données. Sinon les robots, navigateurs et proxies (qui pré-chargent les liens GET pour accélérer la navigation) déclencheront des actions à votre insu.

4.2 POST

Définition

POST : crée une nouvelle ressource ou déclenche une action. Cette méthode n'est *ni sûre ni idempotente*. Elle accepte un corps de requête.

- `POST /articles` avec un corps JSON : créer un article.
- Code de succès attendu : **201 Created** avec un en-tête `Location` pointant la nouvelle ressource.

4.3 PUT

Définition

PUT : remplace *intégralement* une ressource. Méthode idempotente.

`PUT /articles/42` avec le corps complet de la ressource. Les champs absents sont supposés effacés (ou remis à leur valeur par défaut).

4.4 PATCH

Définition

PATCH : modifie *partiellement* une ressource. L'idempotence dépend de l'opération.

`PATCH /articles/42` avec uniquement les champs à modifier. Plus économe et moins risqué que PUT.

4.5 DELETE

Définition

DELETE : supprime une ressource. Méthode idempotente.

`DELETE /articles/42`. Code de succès : **204 No Content** (succès sans corps).

4.6 Tableau récapitulatif

Méthode	Action	Sûre	Idemp.	Code typique
GET	Lire	Oui	Oui	200
POST	Créer / agir	Non	Non	201
PUT	Remplacer entièrem.	Non	Oui	200 / 204
PATCH	Modifier partiel.	Non	Selon	200 / 204
DELETE	Supprimer	Non	Oui	204

4.7 Notions importantes : « sûre » et « idempotente »

Définition

Méthode sûre : qui ne modifie pas l'état du serveur. C'est une garantie sémantique du contrat HTTP. Les méthodes GET, HEAD, OPTIONS sont sûres.

Méthode idempotente : appeler N fois cette méthode produit le même résultat qu'un seul appel. Critique pour la robustesse aux pannes réseau : si le client perd la connexion sans recevoir la réponse, il peut rejouer une requête idempotente sans risque.

5 Les codes de statut HTTP

Chaque réponse HTTP contient un code numérique à trois chiffres. Le premier chiffre indique la *famille*.

5.1 Les 5 familles

- **1xx — Informationnel** : continuation, changement de protocole. Rare en API.
- **2xx — Succès** : la requête a été traitée avec succès.
- **3xx — Redirection** : action supplémentaire nécessaire.
- **4xx — Erreur client** : la requête est invalide ou non autorisée.
- **5xx — Erreur serveur** : le serveur a échoué à traiter une requête pourtant valide.

5.2 Les codes 2xx les plus utilisés

- **200 OK** : succès générique avec corps de réponse.
- **201 Created** : ressource créée. Ajouter `Location: ...`.
- **202 Accepted** : traitement asynchrone démarré.
- **204 No Content** : succès sans corps. Idéal pour DELETE et PUT.

5.3 Les codes 4xx les plus utilisés

- **400 Bad Request** : requête mal formée (JSON invalide, syntaxe).
- **401 Unauthorized** : authentification absente ou invalide. *Mal nommé : signifie « non authentifié ».*
- **403 Forbidden** : authentifié mais sans la permission. À distinguer du 401.
- **404 Not Found** : ressource inexistante.
- **405 Method Not Allowed** : méthode non supportée pour cette URI.
- **409 Conflict** : conflit d'état (doublon, version obsolète).
- **422 Unprocessable Entity** : syntaxe correcte mais validation métier échouée.
- **429 Too Many Requests** : limite de débit dépassée.

5.4 Les codes 5xx

- **500 Internal Server Error** : exception non gérée côté serveur.
- **502 Bad Gateway** : un serveur amont a renvoyé une erreur.
- **503 Service Unavailable** : maintenance ou surcharge.
- **504 Gateway Timeout** : un serveur amont n'a pas répondu à temps.

5.5 Confusion classique : 401 vs 403

À retenir

401 dit : « je ne sais pas qui vous êtes ». Le client peut afficher un écran de connexion.
403 dit : « je sais qui vous êtes et vous n'avez pas le droit ». Pas de connexion à refaire, c'est définitif.

6 Bonnes pratiques de conception

6.1 Nommer les URI

Les URI doivent être lisibles, prévisibles et cohérentes.

- **Pluriel** pour les collections : `/articles`, `/users`, `/orders`.

- **Pas de verbes** : la méthode HTTP est déjà le verbe. Préférer `POST /articles` à `POST /createArticle`.
- **Hiérarchie naturelle** pour les sous-ressources : `/articles/42/comments/7`.
- **Tirets** en kebab-case : `/blog-posts`, pas `/blogPosts` ni `/blog_posts`.
- **Pas d'extension** de format : `/articles`, jamais `/articles.json`. Le format est négocié via l'en-tête `Accept`.

Exemple

À faire : `POST /api/v1/users/42/orders`
À éviter : `POST /api/createOrderForUser?userId=42`

6.2 Versionner l'API

Une API évolue. Pour ne pas casser les clients existants, on conserve l'ancienne version pendant une période de transition.

Trois écoles :

1. Dans l'URI : `/api/v1/articles` puis `/api/v2/articles`. **Le plus utilisé**.
2. Dans un en-tête : `Accept: application/vnd.acme.v2+json`. URI plus propres mais moins lisibles.
3. Dans une query string : `?api-version=2`. Déconseillé, mélange version et filtres.

6.3 Pagination, filtrage, tri

Toute liste potentiellement longue doit être paginée. Conventions courantes :

- Pagination par offset : `?page=2&size=20`.
- Pagination par curseur : `?cursor=eyJ...&limit=20`. Plus stable face aux insertions.
- Tri : `?sort=-date,name` (le `-` signifie descendant).
- Filtres : `?author=42&status=published`.
- Recherche plein texte : `?q=rest`.

6.4 Gérer les erreurs : Problem Details

Le standard pour les erreurs en API REST est défini par la RFC 7807 (renumérotée 9457 en 2023) sous le nom *Problem Details*. Format :

Exemple

```
HTTP/1.1 422 Unprocessable Entity
Content-Type: application/problem+json

{
  "type": "https://api.example.com/errors/validation",
  "title": "Validation échouée",
  "status": 422,
  "detail": "Le champ \"email\" est invalide.",
  "instance": "/articles",
  "traceId": "9f1c-...-7e0a"
}
```

6.5 Anti-patterns à éviter

- Verbes dans l'URI : `/getUser`, `/createOrder`.
- GET avec effet de bord : `GET /articles/42/delete`.
- Retourner `200 OK` avec un champ `error` : casse les outils HTTP.
- Tout retourner en 500, peu importe la nature de l'erreur.
- Versioning oublié, puis breaking change qui casse tous les clients.

7 Les outils du quotidien

7.1 Postman

Outil

Postman est l'outil le plus utilisé pour tester et documenter des API REST. Disponible en version desktop gratuite (Windows, macOS, Linux) et en SaaS.

À quoi ça sert ?

- Construire une requête HTTP visuellement (méthode, URL, headers, body).
- Sauvegarder ses requêtes en *collections* et les partager avec son équipe.
- Définir des *environnements* (dev, staging, prod) avec des variables.
- Écrire des tests automatisés qui valident la réponse.
- Générer du code client (cURL, Python, JavaScript, etc.).

Site officiel. `https://www.postman.com`

7.2 Insomnia

Outil

Insomnia (édité par Kong) est une alternative à Postman, plus légère et orientée développeur. Open source pour la version de base.

Avantages : interface plus épurée, support natif de GraphQL, gestion fine des environnements.

Site officiel. `https://insomnia.rest`

7.3 cURL

Outil

cURL (*Client URL*) est un utilitaire en ligne de commande disponible sur Linux, macOS et Windows. Indispensable pour tester rapidement une API ou pour scripter des appels HTTP.

Exemples :

```
curl https://api.example.com/articles
curl -X POST https://api.example.com/articles \
     -H "Content-Type: application/json" \
     -d '{"title":"Hello"}'
curl -H "Authorization: Bearer xyz" https://api.example.com/me
```

7.4 Swagger / OpenAPI

Outil

OpenAPI (anciennement *Swagger*) est une spécification standard pour décrire une API REST de manière formelle, dans un fichier YAML ou JSON. À partir de cette description, on peut générer automatiquement :

- Une documentation interactive (Swagger UI ou Redoc).
- Des clients SDK dans plusieurs langages.
- Des tests automatisés.
- Du code serveur de base.

Site officiel. https://www.openapis.org

8 Exemple complet : conception d'une API de blog

Mettons en pratique ce qui précède sur un cas concret : une API pour un blog avec articles, auteurs et commentaires.

8.1 Liste des endpoints

Méthode	URI	Action
GET	/api/v1/articles	Lister les articles
POST	/api/v1/articles	Créer un article
GET	/api/v1/articles/{id}	Lire un article
PUT	/api/v1/articles/{id}	Remplacer un article
PATCH	/api/v1/articles/{id}	Modifier un article
DELETE	/api/v1/articles/{id}	Supprimer un article
GET	/api/v1/articles/{id}/comments	Lister les commentaires
POST	/api/v1/articles/{id}/comments	Ajouter un commentaire
DELETE	/api/v1/articles/{aid}/comments/{cid}	Supprimer un commentaire

8.2 Exemple de réponse paginée

```
GET /api/v1/articles?page=2&size=20&sort=-date

{
```

```
  "data": [
    { "id": 21, "title": "...", "author": "Alice", "createdAt": "..." },
    { "id": 22, "title": "...", "author": "Bob", "createdAt": "..." }
  ],
  "pagination": {
    "page": 2, "size": 20, "totalPages": 14, "totalItems": 273
  },
  "_links": {
    "self": "/api/v1/articles?page=2&size=20",
    "next": "/api/v1/articles?page=3&size=20",
    "prev": "/api/v1/articles?page=1&size=20"
  }
}
```

9 Synthèse

À retenir

- REST est un **style architectural**, pas un protocole. Il s'appuie sur HTTP et 6 contraintes.
- Une **méthode HTTP = une intention** : GET lit, POST crée, PUT remplace, PATCH modifie, DELETE supprime.
- Distinguer **sûre** (sans effet de bord) et **idempotente** (N appels = 1 appel).
- Choisir le bon **code de statut** : 2xx succès, 4xx faute du client, 5xx faute du serveur. 401 ≠ 403.
- URI **lisibles et cohérentes** : pluriel, pas de verbes, hiérarchie naturelle, query string pour filtres.
- Format d'erreur unique : **Problem Details** (RFC 7807).

10 Lexique du module

API Application Programming Interface : contrat permettant à des logiciels de communiquer.

Cache Stockage temporaire d'une réponse pour la réutiliser sans la recalculer.

CRUD Create, Read, Update, Delete — les quatre opérations de base sur une ressource.

ETag Identifiant de version d'une ressource, utile pour le cache HTTP.

HTTP HyperText Transfer Protocol — le protocole de communication du web.

Idempotent Une opération qui produit le même résultat quel que soit le nombre d'appels.

JSON JavaScript Object Notation — format de données structurées le plus utilisé en API REST.

OpenAPI Spécification standard pour décrire formellement une API REST (anciennement Swagger).

Pagination Découpage d'une liste longue en plusieurs pages successives.

Problem Details Format standardisé pour les réponses d'erreur (RFC 7807/9457).

REST REpresentational State Transfer — style architectural basé sur HTTP.

Ressource Tout ce que l'API expose (utilisateur, article, commande...).

Sûre Qualifie une méthode HTTP qui ne modifie pas l'état du serveur.

URI Uniform Resource Identifier — chaîne identifiant une ressource.

Versioning Conservation de plusieurs versions d'une API pour ne pas casser les clients existants.

11 Pour aller plus loin

- Roy Fielding, *Architectural Styles and the Design of Network-based Software Architectures*, thèse de doctorat, 2000.
- Leonard Richardson & Mike Amundsen, *RESTful Web APIs*, O'Reilly, 2013.
- RFC 9110 (HTTP Semantics) — la référence unique pour la sémantique HTTP.
- RFC 7807 / 9457 (Problem Details).
- Sites de référence : `https://restfulapi.net`, `https://www.openapis.org`.